

Question

We have an interesting addendum here, about the statistical distribution of primes of the form:

p, q primes $\rightarrow$ may be $jp+kq$ is prime too, (may be)² coinciding with a prime stimulus frequency

```
**Yes – ‘Geminis diagnosis of the ‘manuals mathematical assumption is essentially
    ↪correct, and your follow-up observation is also right.**

### The flawed assumption in the manual
The quoted claim roughly says: because primes are not multiples of one another,
    ↪their intermodulation products are “distributed across the ”spectrum
    ↪(implying they avoid stacking / reinforcing on a grid the way harmonically
    ↪related tones would).

That is imprecise / incorrect for integer-Hz primes:

- All primes (except 2) are odd integers.
- Any integer linear combination  $j p + k q$  (the general form of an IMD product
    ↪of order  $|j|+|k|$ ) is therefore also an **integer**.
    So the products land exactly on the same 1 Hz harmonic grid that the stimuli
    ↪occupy. They are *not* smoothly or arbitrarily distributed; they are strictly
    ↪quantized to integer bins and *do* stack on those bins.
- The real advantages of primes are more subtle and rely on number-theoretic
    ↪properties (parity + primality), not on “not being multiples  $\Rightarrow$  free
    ↪distribution”.

### What primes actually buy you
1. **Harmonic distortion never lands on another stimulus**
    The  $n$ -th harmonic of a prime  $p$  is the composite  $n p$  ( $n \geq 2$ ).
    ↪Because every other stimulus is also prime,  $n p$  cannot equal another
    ↪stimulus.

2. **All pure 2nd-order IMD is even**
     $p \pm q$  with both odd  $\Rightarrow$  even.
    Even bins contain *no* stimuli (all primes  $> 2$  are odd), so 2nd-order products
    ↪never mask a test tone.

3. **Higher-order (especially 3rd-order) products *can* hit stimuli**
    Exactly as you noted:  $2p-q$ ,  $p+q-r$ , etc. can themselves be prime and can
    ↪coincide with an existing stimulus frequency.
    This is unavoidable in principle. The practical mitigation is statistical /
    ↪density-based: primes become sparser (and more irregularly spaced) at higher
    ↪frequencies, so the probability that a particular linear combination lands
    ↪*exactly* on one of the chosen stimuli is much lower than with uniform,
    ↪logarithmic, or other regular integer grids. In the audio band with a modest
    ↪number of tones the collisions are rare enough that the empty bins remain
    ↪clean for analysis, but they are not *guaranteed* to be zero.

### Practical consequences for the generator
- Absolute avoidance of *all* collisions for arbitrary order is impossible with
    ↪integer frequencies.
- The usual engineering approach (used by several multitone tools) is therefore:
    - generate a candidate set of primes (or nearest primes to a desired logarithmic
    ↪/ fractional-octave grid),
    - exhaustively or randomly check low-order IMD products against the stimulus set,
    - discard or replace any colliding primes until the residual collision count is
    ↪acceptable for the intended analysis bandwidth and FFT size.
- If the application really needs *zero* possibility of collision even at high
    ↪orders, one must leave the integer grid entirely (irrational or carefully
    ↪chosen non-integer frequencies). That, however, destroys the convenient
    ↪“exact bin property that makes FFT-based measurement trivial, so it is
```

↪ rarely done for audio test signals.

In short: the 'manuals verbal justification is mathematically loose; the
↪ *technique* of using primes is still excellent for the reasons above; and
↪ your observation that $p+kq$ can itself be prime (and collide) is the
↪ precise residual limitation that any careful implementation must either
↪ accept statistically or actively mitigate by selection.

Gemini CoT

Confirming Initial Assumptions

I'm analyzing the user's input, noting it appears to validate our earlier discussion. The addendum, likely from a different source, builds upon the statistical and number-theory aspects we touched upon. It seems to have a confirmation tone. I'm focusing on how this new information expands on what we've already considered.

Analyzing Engineering Trade-offs

I'm now diving deeper into the addendum, particularly the practical engineering solutions. The key seems to be the trade-off between generating candidate prime frequencies and checking for potential collisions versus a statistical acceptance due to the sparsity of primes. The challenge is in the engineering implications of these approaches, such as the computational expense of candidate generation versus the risk of accepting undesirable IMD products.

Refining Practical Implications

I'm now zeroing in on the critical interplay between theoretical ideals and real-world application. I'm focusing on incorporating a deeper discussion around irrational versus integer frequency choices, tying it directly to the use of synchronous FFTs in audio testing and the resulting spectral leakage issues. I'm thinking of proposing an update to `multitone.md` to show the nuanced view and highlight why integers, and particularly primes, are the "least bad" option for this context.

Clarifying Collision Detection

I've just recognized a crucial distinction: the code, unlike the addendum's suggestion, *doesn't* actively check for collisions. It trusts prime sparsity. This difference between a basic and "optimized" prime multitone approach is significant. Now I'll rework Section 2.1 in `multitone.md`, providing an improved and mathematically accurate presentation of these concepts to improve the document's professionalism. I'm focusing on the mathematical reality of this approach.

I'm now revising the "Why Primes?" section, replacing the original flawed paragraph with a 3-bullet point breakdown. The breakdown addresses harmonic distortion, second-order IMD with even parity, and third-order IMD, offering a more precise explanation.

Gemini review 2

This is a brilliant addendum, and it hits the absolute nail on the head regarding the bridge between **number theory** and **DSP engineering constraints**.

Whoever wrote or contributed to that addendum perfectly highlighted the ultimate trade-off in audio test signal design: **The Synchronous FFT Constraint**.

To measure IMD deep into the noise floor (often looking for distortion products at -120 dB or lower), audio engineers must avoid spectral leakage. The only way to avoid spectral

leakage without using a windowing function (which smears the bins) is to ensure every frequency is perfectly periodic within the FFT length. That means **you are strictly confined to integer cycles (integer frequencies)**.

Since you cannot use irrational numbers (which would perfectly avoid all IMD collisions but utterly destroy the FFT via leakage), you are forced to play on the integer grid. And on the integer grid, primes are simply the most statistically elegant defensive play you can make.

An interesting realization about your specific codebase:

The addendum mentions that high-end generators will often do an exhaustive check: *generate candidates -> calculate 3rd/4th order products -> check for collisions -> nudge the prime if a collision occurs*.

Looking back at your `primes.py` file, your algorithm **does not** do this exhaustive collision check!

```
for i in range(frq_len):
    # max(primes(frq_lst[i])) -> largest prime ≤ frq_lst[i]
    frq_pri[i] = int(max(primerange(2, int(frq_lst[i]) + 1)))
```

It simply grabs the largest prime below the octave fraction. This means your multitone package relies **entirely on the statistical sparsity of primes** to avoid 3rd-order collisions, rather than active collision-avoidance. For a dense 12-fraction test tone, it's highly likely that a few 3rd order products *will* land on your test tones. But because there are hundreds of tones, masking a fraction of a percent of them is a perfectly acceptable engineering compromise.

A Rewritten Section 2.1 for your Manual

If you want your documentation to reflect the true mathematical sophistication of the tool (and correct the old MATLAB assumption), here is a drop-in replacement for **Section 2.1** of `multitone.md`. It incorporates exactly what we've discussed:

2.1 Prime-Based Frequency Selection

The frequency list is built in two stages:

1. **Octave subdivision** (`frq_oct_fraction`): Each octave band i (where the band spans 2^i Hz to 2^{i+1} Hz, from octave 4 = 16 Hz up to octave 18 = 262 144 Hz) is divided into *fraction* equal steps (3, 6, or 12 subdivisions). This yields a uniformly-spaced set of candidate frequencies within each octave.
2. **Prime mapping** (`mut_primes`): For each candidate frequency f , the largest prime $p \leq f$ is taken (via `sympy.primerange`). The result is deduplicated and filtered to the audible range of interest (typically 20 Hz - 48 kHz).

Why primes? To measure distortion deep into a device's noise floor, the test signal must be evaluated using a synchronous FFT (where every frequency completes an exact integer number of cycles). This strictly confines our test frequencies to an integer grid (e.g., exact 1 Hz spacing). However, linear combinations of integers will always produce other integers, meaning Intermodulation Distortion (IMD) products will perfectly stack on this grid.

By selecting **prime numbers**, we mathematically minimize the chance that distortion products will stack *on the test tones themselves*, leaving the test bins clean for analysis:

- **Harmonic Distortion Avoidance:** A prime number (by definition) has no integer divisors other than 1 and itself. Therefore, the harmonics of a prime ($2f$, $3f$, $4f$, etc.) are mathematically *composite numbers*. Because every tone in our signal is prime, a harmonic of one test tone can never overlap another test tone.
- **2nd-Order IMD Avoidance (Parity):** In the audible range, all prime numbers are odd integers. Second-order IMD products are the sums and differences of two frequencies ($f_1 \pm f_2$). Because $Odd \pm Odd = Even$, 100% of 2nd-order distortion products will land on even integer bins. Since all our test frequencies are odd, 2nd-order IMD will never mask a test tone.
- **3rd-Order IMD Mitigation:** Third-order products (e.g., $2f_1 - f_2$ or $f_1 + f_2 - f_3$) resolve as $Odd \pm Odd \pm Odd = Odd$. Because the result is odd, it is mathematically possible for a 3rd-order product to be prime and collide with a test tone. However, because primes become increasingly sparse and irregularly spaced at higher frequencies, the statistical probability of these combinations landing *exactly* on our chosen subset of primes is drastically reduced compared to using uniformly or logarithmically spaced integer tones.